

Internet Applications Design and Implementation

Week 7 - Dynamic front-end in SSR. Automated Tests.

Informations

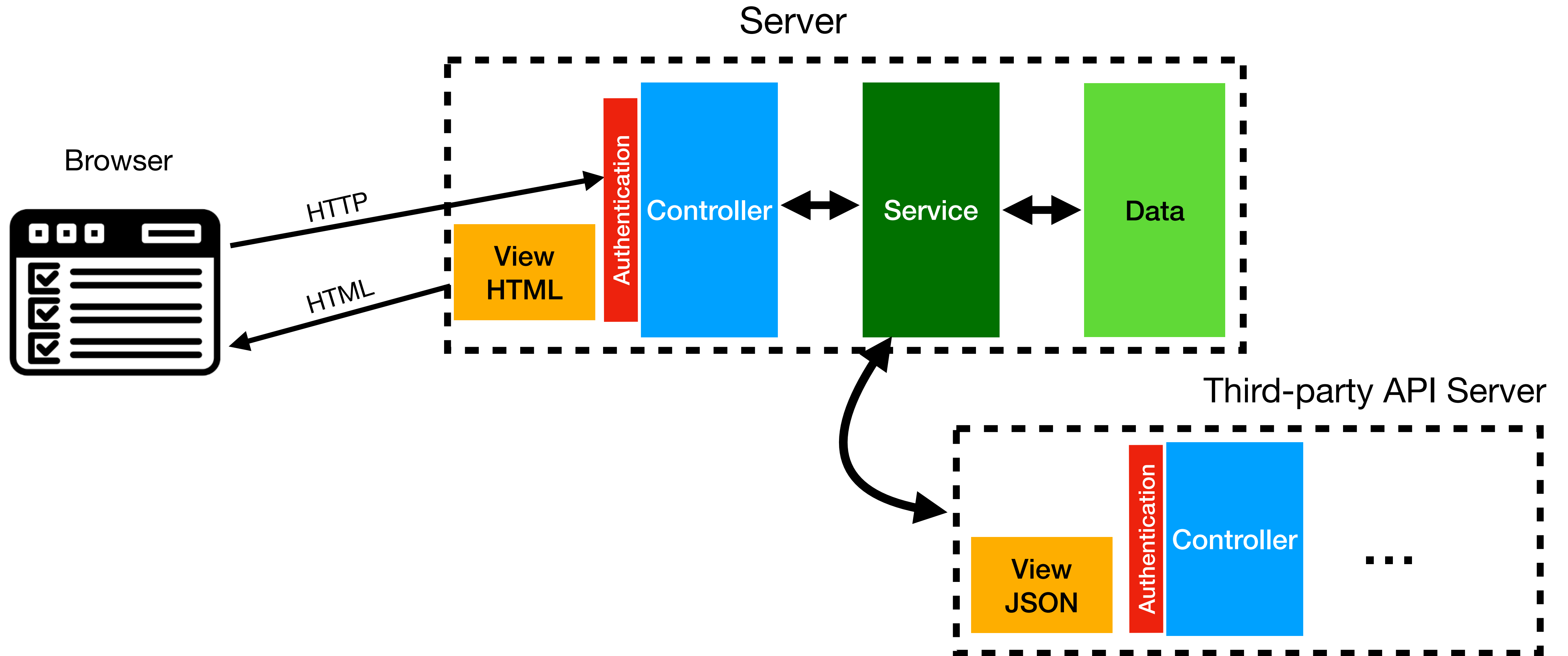
Instructions for the Project (part 2) have been published

1. Start by creating a maven multi-module project
2. Identify the microservices that will be needed to implement the mandatory user stories
3. Implement each one as a standalone REST API application (similar to the bookstore assignment)
4. Next week, you'll learn about how to integrate them in a microservices architecture

Enrollment for Test 1 is now open (until 22 April)

1. Next week I will talk about the Test structure

Consuming APIs in SSR



Spring Consuming APIs

Spring offers several options for implementing REST clients:

- **RestClient** - Synchronous HTTP client with a fluent API
- **WebClient** - Non-blocking client with a fluent API, suitable for async and streaming scenarios
- **RestTemplate** - Deprecated, still widely used, should be replaced by RestClient or WebClient
- **@HttpExchange interfaces** - Declarative-style client with strong typing (built on top of RestClient)
- **OpenFeign** - Declarative-style client integrated into Spring Cloud

Spring RestClient

```
@Value("\${weather.api.key}") lateinit var apiKey: String  
  
fun fetchWeather(location: String): WeatherResponse? {  
    val client: RestClient = RestClient.builder()  
        .baseUrl("https://api.openweathermap.org")  
        .build()  
  
    return client.get()  
        .uri {  
            it.path("/data/2.5/weather")  
                .queryParams("q", location)  
                .queryParams("appid", apiKey)  
                .queryParams("units", "metric")  
                .build()  
        }  
        .retrieve()  
        .body<WeatherResponse>()  
}
```

← Never hardcode API keys!

← This client object can be reused across several requests

← Also allows post(), patch(), put(), ...

← This automatically encodes query params (Vila Real → Vila%20Real)

← Unmarshalls JSON response into a DTO

Spring RestClient

```
data class WeatherCondition(val main: String, val description: String)
data class WeatherResponse(val name: String, val weather: List<WeatherCondition>)
```

```
{
  "weather": [
    {
      "id": 800,
      "main": "Clear",
      "description": "clear sky",
      "icon": "01d"
    }
  ],
  "timezone": 3600,
  "id": 2267057,
  "name": "Lisbon",
  "cod": 200
}
```

JSON attributes must match DTO attributes

By default, extra attributes will be ignored. You can override this behaviour:

```
spring.jackson.serialization.fail-on-unknown-properties=true
```

(doesn't mean you should)

Spring RestClient

```
fun fetchWeather(location: String): WeatherResponse? {  
    ...  
  
    return client.get()  
        ...  
        .retrieve()  
        .body<Map<String, Any>>()  
}
```

You can also unmarshal to a (generic) Map

- Loose type safety
- Have to cast to access values

Spring RestClient

```
fun registerProduct(product: ProductRequest): ProductResponse? {  
    val client: RestClient = RestClient.builder()  
        .baseUrl("https://api.example.com")  
        .build()  
  
    return client.post()  
        .uri {  
            it.path("/products")  
            .build()  
        }  
        .headers { headers ->  
            headers.set("Authorization", "Bearer $token")  
        }  
        .body(product)  
        .retrieve()  
        .body(ProductResponse::class.java)  
}
```

Example of a POST
request with an
authorization header

@HttpExchange

```
interface OpenWeatherClient {  
    @GetExchange("/data/2.5/weather")  
    fun getWeather(  
        @RequestParam q: String,  
        @RequestParam appid: String,  
        @RequestParam units: String  
    ): WeatherResponse  
}
```

Spring HttpExchange interface

Declare the interface, Spring will handle the implementation

Notice it uses RestClient which
must be injected for testability

@Configuration

```
class OpenWeatherClientConfig(private val builder: RestClient.Builder) {  
    @Bean  
    fun openWeatherClient: OpenWeatherClient {  
        val restClient = builder  
            .baseUrl("https://api.openweathermap.org")  
            .build()  
        val factory = HttpServiceProxyFactory.builderFor(RestClientAdapter.create(restClient)).build()  
        return factory.createClient(OpenWeatherClient::class.java)  
    }  
}
```

Declare the client
object as a bean

Injected by Spring when needed

@Controller

```
class WeatherController(private val openWeatherClient: OpenWeatherClient) {  
  
    @GetMapping("/weather")  
    fun getWeather() {  
        val weather = openWeatherClient.getWeather(q = "Lisbon", appid = apiKey, units = "metric")  
        // ...  
    }  
}
```

Spring HttpExchange interface

```
@HttpExchange
interface OpenWeatherClient {
    @GetExchange("/data/2.5/weather")
    fun getWeather(
        @RequestParam q: String,
        @RequestParam units: String,
        @RequestHeader("Authorization") authorization: String
    ): OpenWeatherResponse
}
```

Set headers using
@RequestHeader



```
@HttpExchange
interface ProductsClient {
    @GetExchange("/products/{id}")
    fun getProduct(
        @PathVariable id: Long
    ): ProductResponse
}
```

Set path variables



Spring OpenFeign

```
@SpringBootApplication
@EnableFeignClients ←
class MyApplication

fun main(args: Array<String>) {
    runApplication<MyApplication>(*args)
}
```

```
@FeignClient(name = "openweather",
            url = "https://api.openweathermap.org")
interface OpenWeatherFeignClient {
    @GetMapping("/data/2.5/weather")
    fun getWeather(
        @RequestParam q: String,
        @RequestParam appid: String,
        @RequestParam units: String
    ): WeatherResponse
}
```

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
  <version>4.3.1</version>
</dependency>
```

Similar to HttpExchange, declare the interface, OpenFeign will handle the implementation

Injected by Spring when needed

```
@Controller
class WeatherController(private val openWeatherClient: OpenWeatherFeignClient) {

    @GetMapping("/weather")
    fun getWeather() {
        val weather = openWeatherClient.getWeather(q = "Lisbon", appid = apiKey, units = "metric")
        // ...
    }
}
```


Spring OpenFeign

```
@FeignClient(name = "openweather",  
    url = "https://api.openweathermap.org")  
interface OpenWeatherFeignClient {  
    @GetMapping("/data/2.5/weather")  
    fun getWeather(  
        @RequestParam q: String,  
        @RequestParam appid: String,  
        @RequestParam units: String,  
        @RequestHeader("Authorization") authorization: String  
    ): WeatherResponse  
}
```

← You can omit the url, if using
service discovery
(more on that in the next lesson)

← Set headers

```
@FeignClient(name = "products",  
    url = "https://api.products.org")  
interface ProductsClient {  
    @PostMapping("/products")  
    fun createFlight(@RequestBody request: CreateProductRequest): ProductDto  
}
```

← Set the POST payload

Spring OpenFeign

By default, only supports GET and POST.

To support the other verbs, add this dependency:

```
<dependency>  
  <groupId>io.github.openfeign</groupId>  
  <artifactId>feign-okhttp</artifactId>  
</dependency>
```

And add this to application.properties:

```
spring.cloud.openfeign.okhttp.enabled=true
```

Spring REST Client

Failure handling (for RestClient and HttpExchange):

- Timeout / no connectivity - ResourceAccessException
- 4xx - HttpClientErrorException
- 5xx - HttpServerErrorException

Failure handling (for OpenFeign):

- Timeout / no connectivity - RetryableException
- 4xx - FeignClientException
- 5xx - FeignServerException

Spring REST Clients

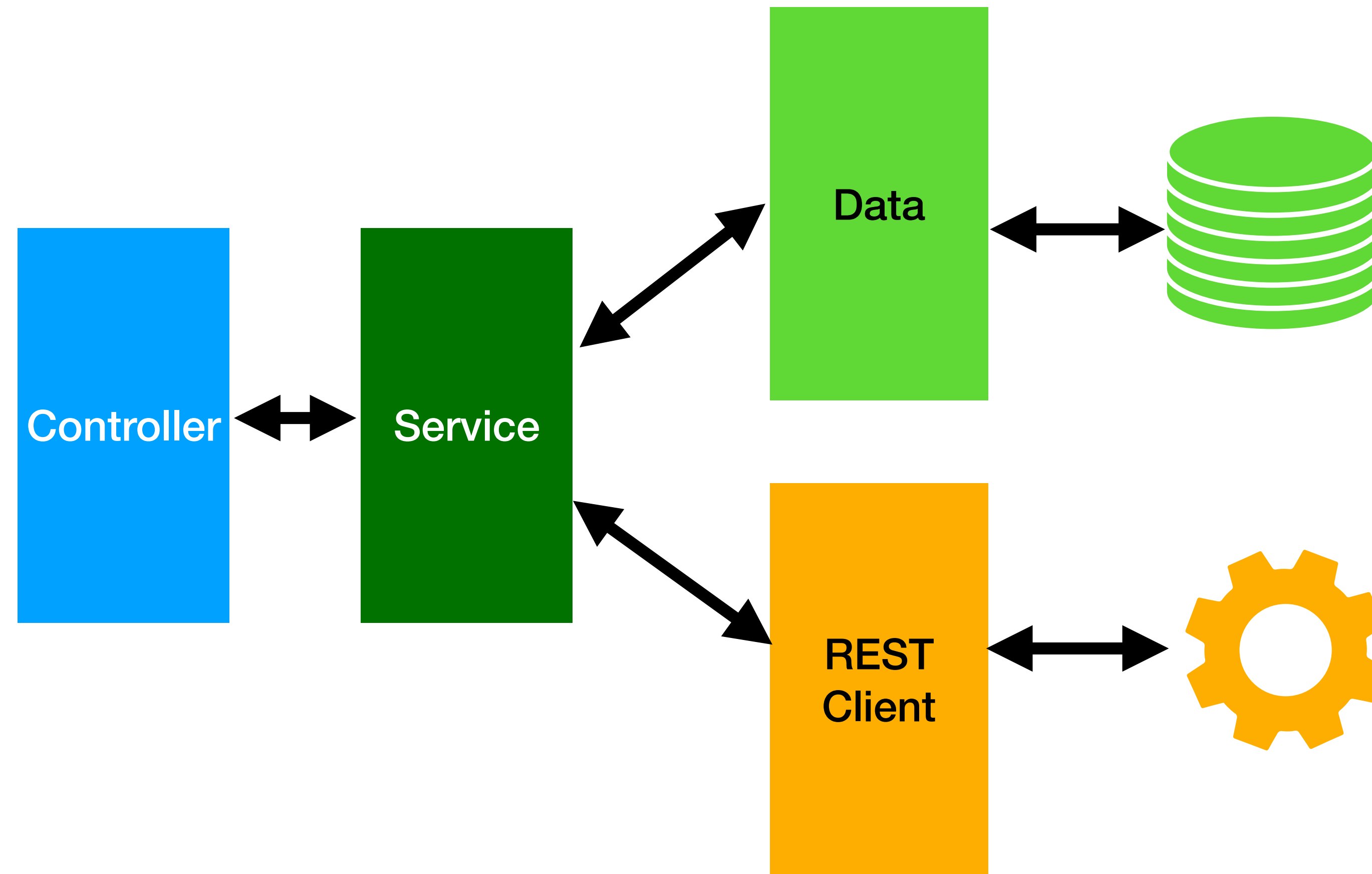
Exercise

	RestClient	HttpExchange	FeignClient
Style (Imperative/Declarative)			
Needs external dependency (yes/no)			
Who is responsible for implementing? (programmer/framework)			
Control/transparency (high, medium, low)			
Best for			

Spring REST Clients

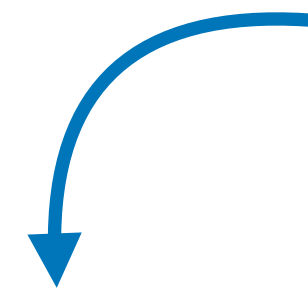
	RestClient	HttpExchange	FeignClient
Style (Imperative/Declarative)	Imperative	Declarative	Declarative
Needs external dependency (yes/no)	no	no	yes
Who is responsible for implementing? (programmer/framework)	programmer	framework	framework
Control/transparency (high, medium, low)	high	medium	low
Best for	One-off calls, full control	Clean API clients	Microservices

Architecture



Architecture

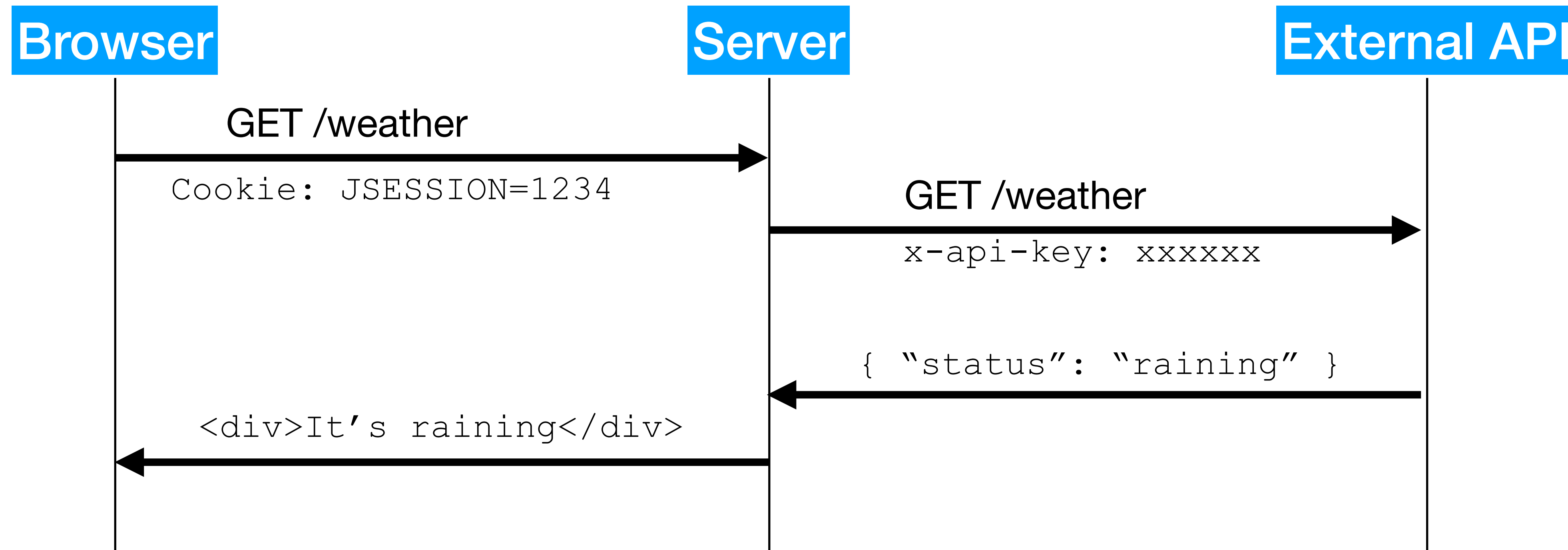
Inject the REST client



```
@Service
class WeatherService(val openWeatherClient: OpenWeatherClient) {

    fun getWeather(location: String): String? {
        openWeatherClient.getWeather(...)
    }
}
```

Architecture

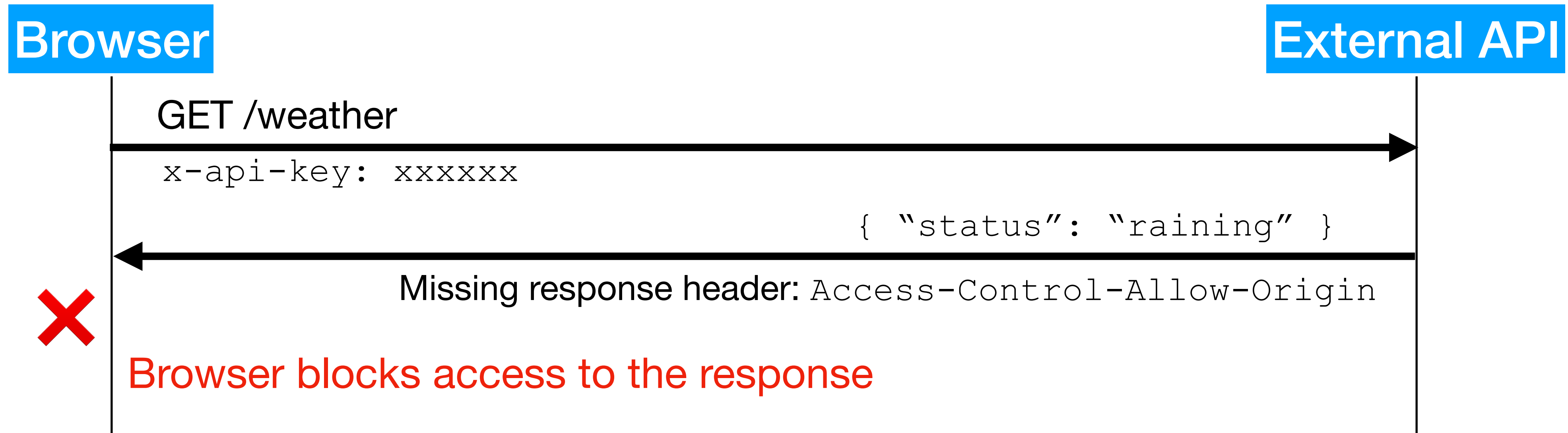


Architecture



Why not go direct to the External API?

Architecture

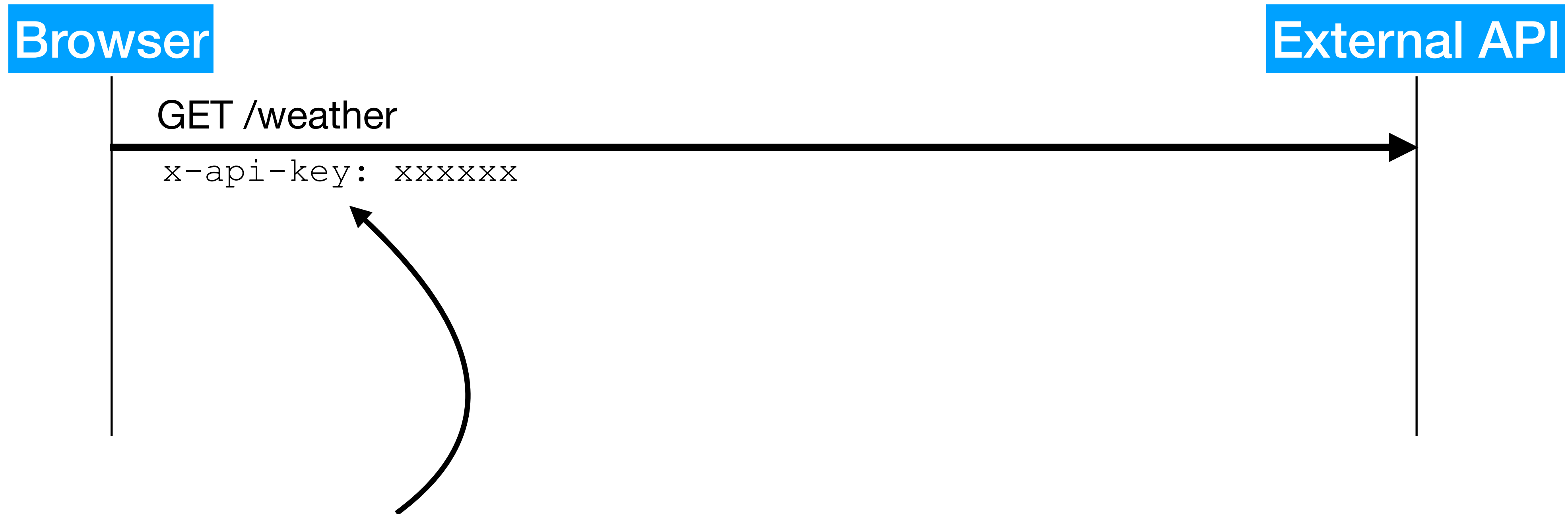


1. Several External APIs block calls from other sites

GET `https://weather-api.com/weather`
Origin: `https://your-app.com`

More info: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>

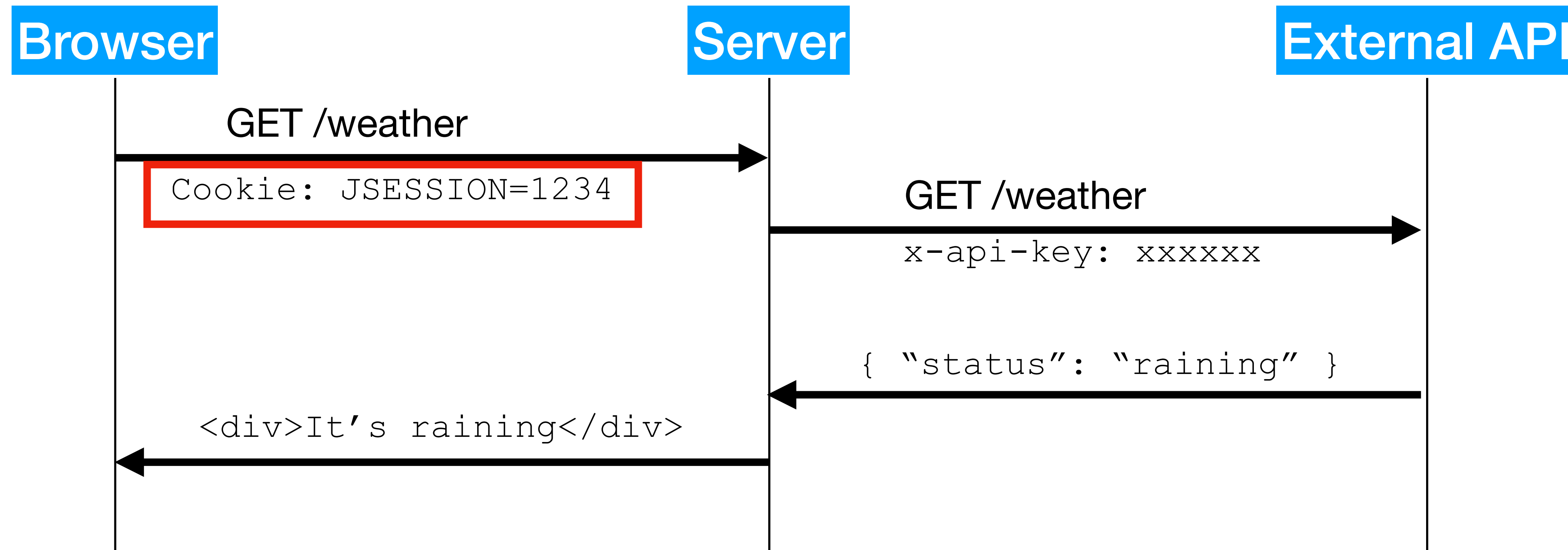
Architecture



2. The API key is exposed!

```
fetch("https://weather-api.com/weather", {
  headers: {
    "x-api-key": "xxxxxxx"
  }
})
```


Architecture



Avoid exposing these endpoints without authentication, to prevent API abuse

Logging / Observability

Best practice: log all API requests and response codes

```
2026-04-07T13:58:30 ... [alice] [External API] GET https://api.openweathermap.org/data/2.5/weather?
q=Lisboa&units=metric [200 OK]
```

Logging - RestClient

```
val restClient = builder
    .baseUrl("https://api.someserver.com")
    .requestInterceptor { request, body, execution ->
        val principal = SecurityContextHolder.getContext().authentication?.name
            ?: "anonymous"

        val response = execution.execute(request, body)
        log.info("[{}] [External API] {} {} [{}]", principal,
            request.method, request.uri, response.statusCode)
        response
    }
    .build()
```

Logging - HttpExchange

```
@Configuration
class OpenWeatherClientConfig(
    private val builder: RestClient.Builder
) {
    private val log = LoggerFactory.getLogger(OpenWeatherClientConfig::class.java)
    @Bean
    fun openWeatherClient(builder: RestClient.Builder): OpenWeatherClient {
        val restClient = builder
            .baseUrl("https://api.openweathermap.org")
            .requestInterceptor { request, body, execution ->
                val principal = SecurityContextHolder.getContext().authentication?.name
                    ?: "anonymous"

                val response = execution.execute(request, body)
                log.info("[{}] [External API] {} {} [{}]", principal,
                    request.method, request.uri, response.statusCode)
                response
            }
            .build()
        val factory = HttpServiceProxyFactory.builderFor(RestClientAdapter.create(restClient)).build()
        return factory.createClient(OpenWeatherClient::class.java)
    }
}
```

Since HttpExchange uses RestClient internally, it's the same technique

Logging - OpenFeign

```
@Component
class FeignLoggingLogger : Logger() {

    private val log = LoggerFactory.getLogger(FeignLoggingLogger::class.java)

    override fun log(configKey: String, format: String, vararg args: Any) {
        // no-op – we produce our own log lines in logAndRebufferResponse
    }

    override fun logAndRebufferResponse(configKey: String, logLevel: Level, response: Response, elapsedTime: Long)
        : Response {
        val request = response.request()
        val principal = SecurityContextHolder.getContext().authentication?.name ?: "anonymous"
        log.info("[{}] [External API] {} {} [{}]", principal, request.httpMethod(), request.url(), response.status())
        return response
    }

    override fun logIOException(configKey: String, logLevel: Level, ioe: IOException, elapsedTime: Long)
        : IOException {
        val principal = SecurityContextHolder.getContext().authentication?.name ?: "anonymous"
        log.error("[{}] [External API] {} failed after {}ms: {}", principal, configKey, elapsedTime, ioe.message)
        return ioe
    }
}
```

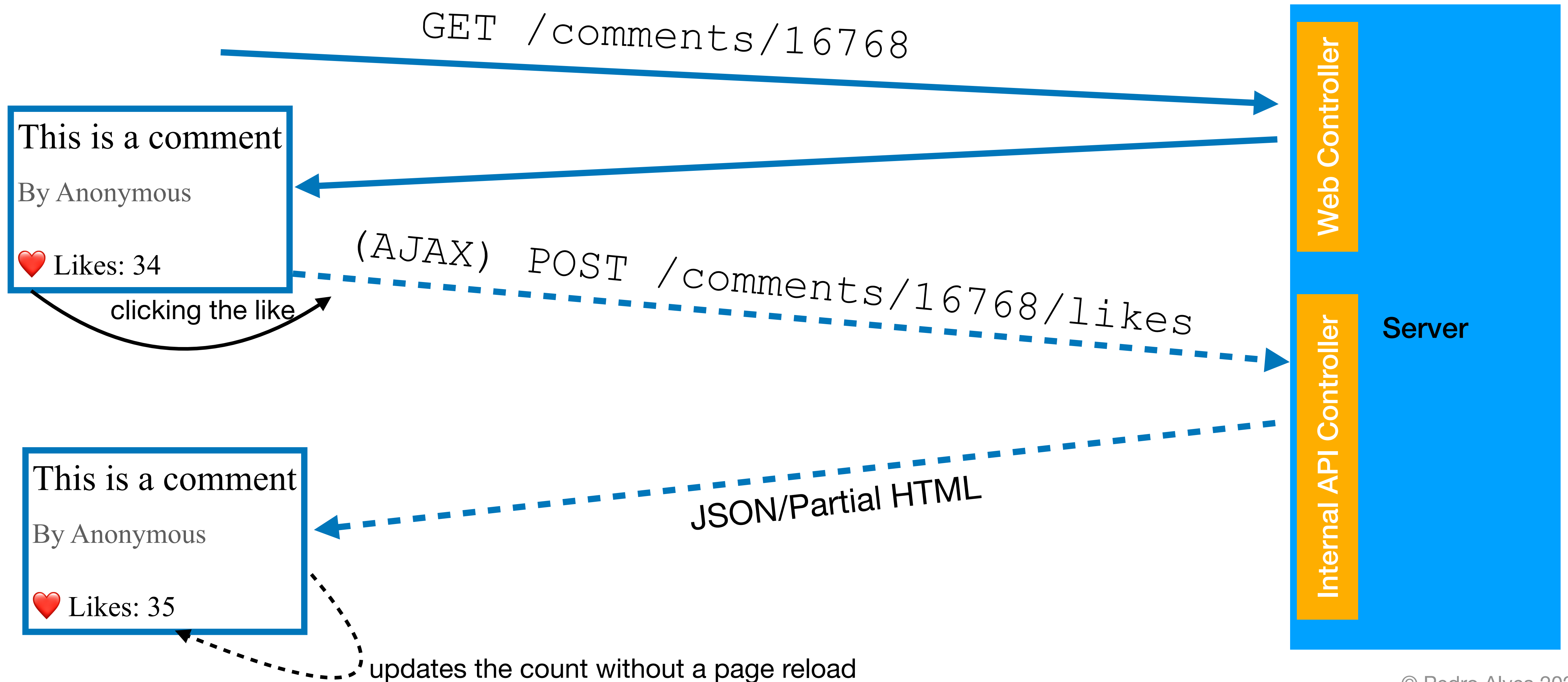
Dynamic frontend in SSR

Typically, in SSR all interactions result in full-page reloads

But you can add some interactivity without fully switching to a CSR model

- Searching, filtering, sorting (without reloads)
- Inline form validation (e.g., email already taken)
- Social interactions (e.g., clicking a like)
- ...

Dynamic frontend in SSR



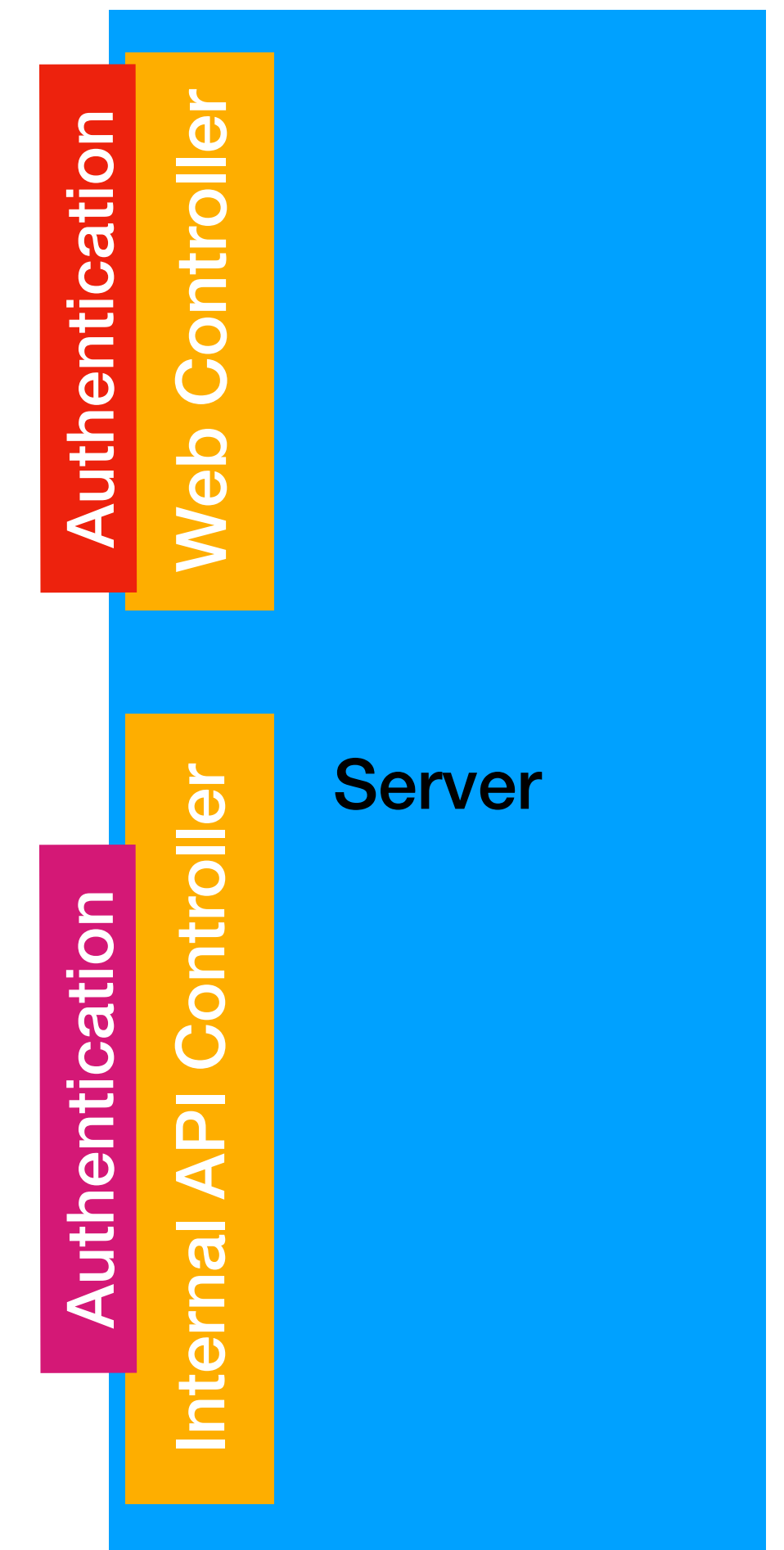
Multiple security chains

Security chain for web controller

- Protect against CSRF attacks
- Form-based login
- Cookie-based session
- 401/403 errors should return HTML

Security chain for Internal API controller

- Token-based authentication
- Cross-origin calls protection (CORS)
- 401/403 errors should return JSON



Multiple security chains

Security chain for web controller

- Protect against CSRF attacks
- Form-based login
- Cookie-based session
- 401/403 errors should return HTML

Security chain for Internal API controller

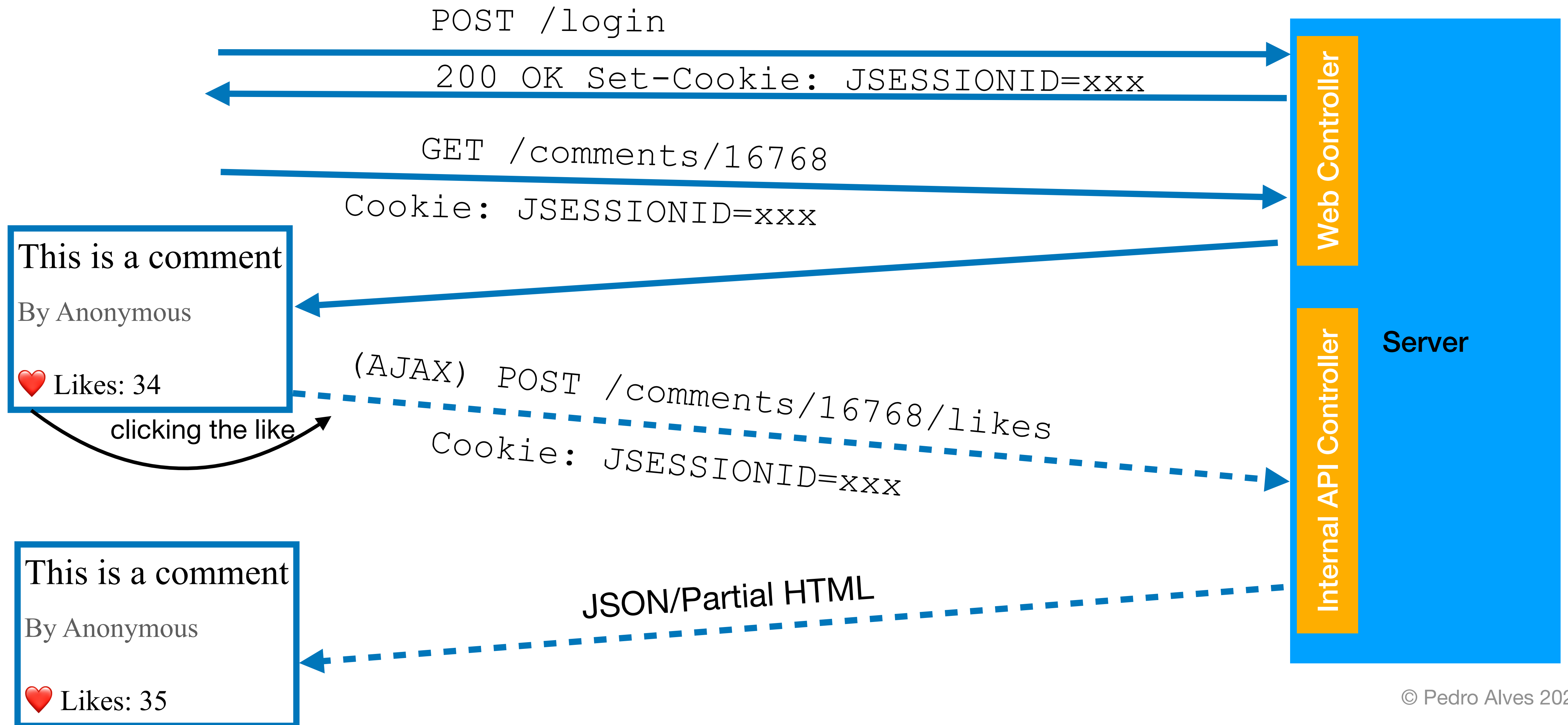
- ~~Token-based authentication~~
- Cross-origin calls protection (CORS)
- 401/403 errors should return JSON

For Internal APIs there is no need for token-based authentication

AJAX/Fetch requests already include the session/JWT cookie

Beware that relying on cookies implies that POST/PUT/DELETE are now prone to CSRF attacks

Session handling for internal APIs



Multiple security chains

SecurityConfig

```
@Bean
@Order(1)
fun internalApiSecurityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        .securityMatcher("/api/**")
        ...
    return http.build()
}
```

```
@Bean
@Order(2)
fun webSecurityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        ...

    return http.build()
}
```

Internal API security chain

@Bean

@Order(1)

```
fun internalApiSecurityFilterChain(http: HttpSecurity): SecurityFilterChain {
```

```
    http
```

```
        .securityMatcher("/api/**")
```

Intercepts only /api requests

```
        .csrf { it.disable() }
```

Must disable CSRF protection (*)

```
        .authorizeHttpRequests { it.anyRequest().authenticated() }
```

```
        .exceptionHandling {
```

```
            it.authenticationEntryPoint { _, res, _ ->
```

```
                res.status = HttpServletResponse.SC_UNAUTHORIZED
```

```
                res.contentType = MediaType.APPLICATION_JSON_VALUE
```

```
                res.writer.write("""{"error": "Unauthorized"}""")
```

```
            }
```

```
            it.accessDeniedHandler { _, res, _ ->
```

```
                res.status = HttpServletResponse.SC_FORBIDDEN
```

```
                res.contentType = MediaType.APPLICATION_JSON_VALUE
```

```
                res.writer.write("""{"error": "Forbidden"}""")
```

```
        }
```

```
    }
```

```
    return http.build()
```

```
}
```

Must define error handling
(return HTTP code and
JSON responses)

By default, cross-origin requests are not allowed

Web security chain

```
@Bean
@Order(2)
fun webSecurityFilterChain(http: HttpSecurity): SecurityFilterChain {
    http
        .authorizeHttpRequests { auth ->
            auth
                ...
                .requestMatchers(HttpMethod.GET, "/clubs/**").permitAll()
                .anyRequest().authenticated()
            }
        .formLogin { form ->
            form.loginPage("/login").permitAll()
        }
    return http.build()
}
```

Catches everything that was not caught by the Internal API Filter

CSRF protection is enabled by default, no need to declare it

Form-based login

No need for explicit error handling:
401 redirects to /login
403 redirects to /error/403.html

CORS - Cross-Origin Resource Sharing

Browser security mechanism that controls:

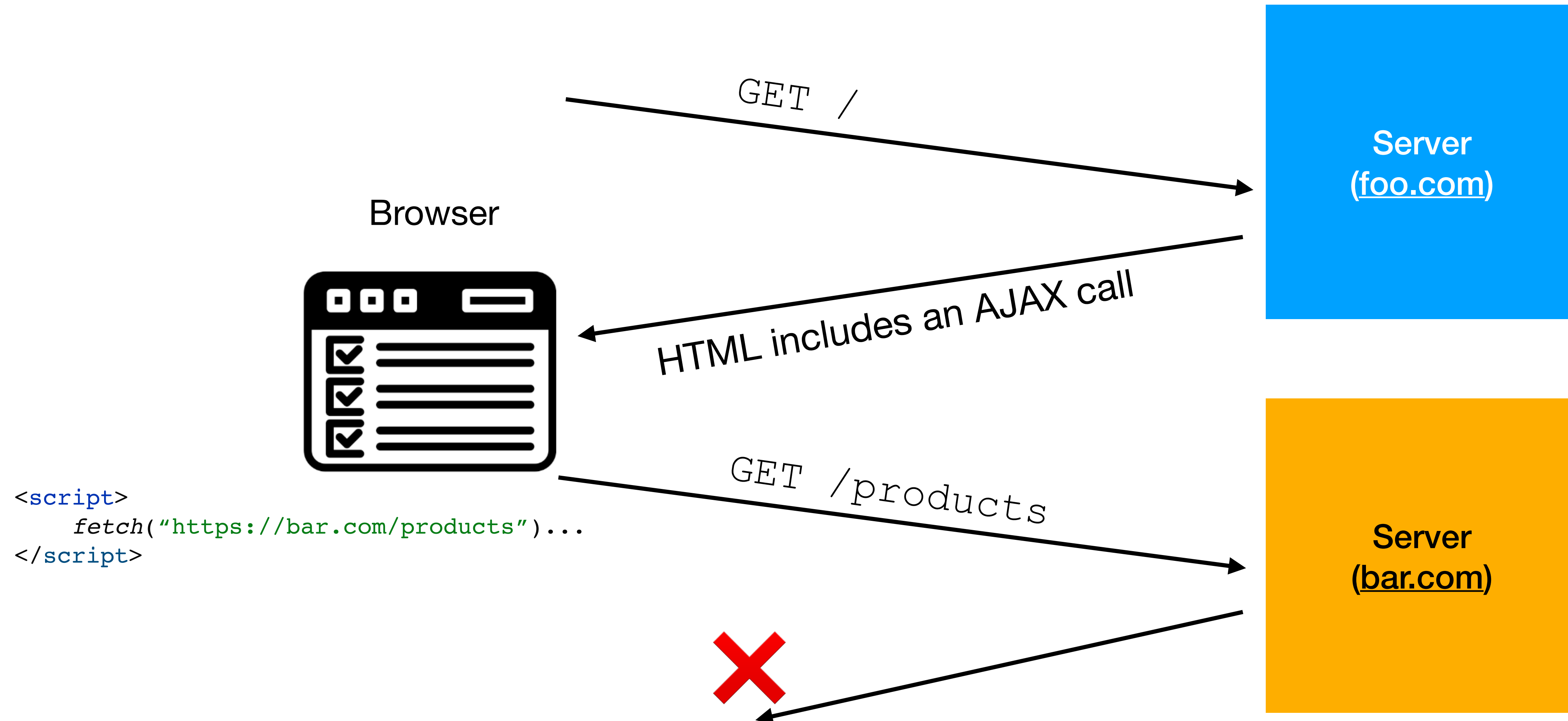
“Can JavaScript on one website read responses from another website?”

- Browsers **allow** sending most cross-origin requests
- Browsers **restrict** reading the response unless allowed

Server may allow requests from certain sites (or all of them), using this response header

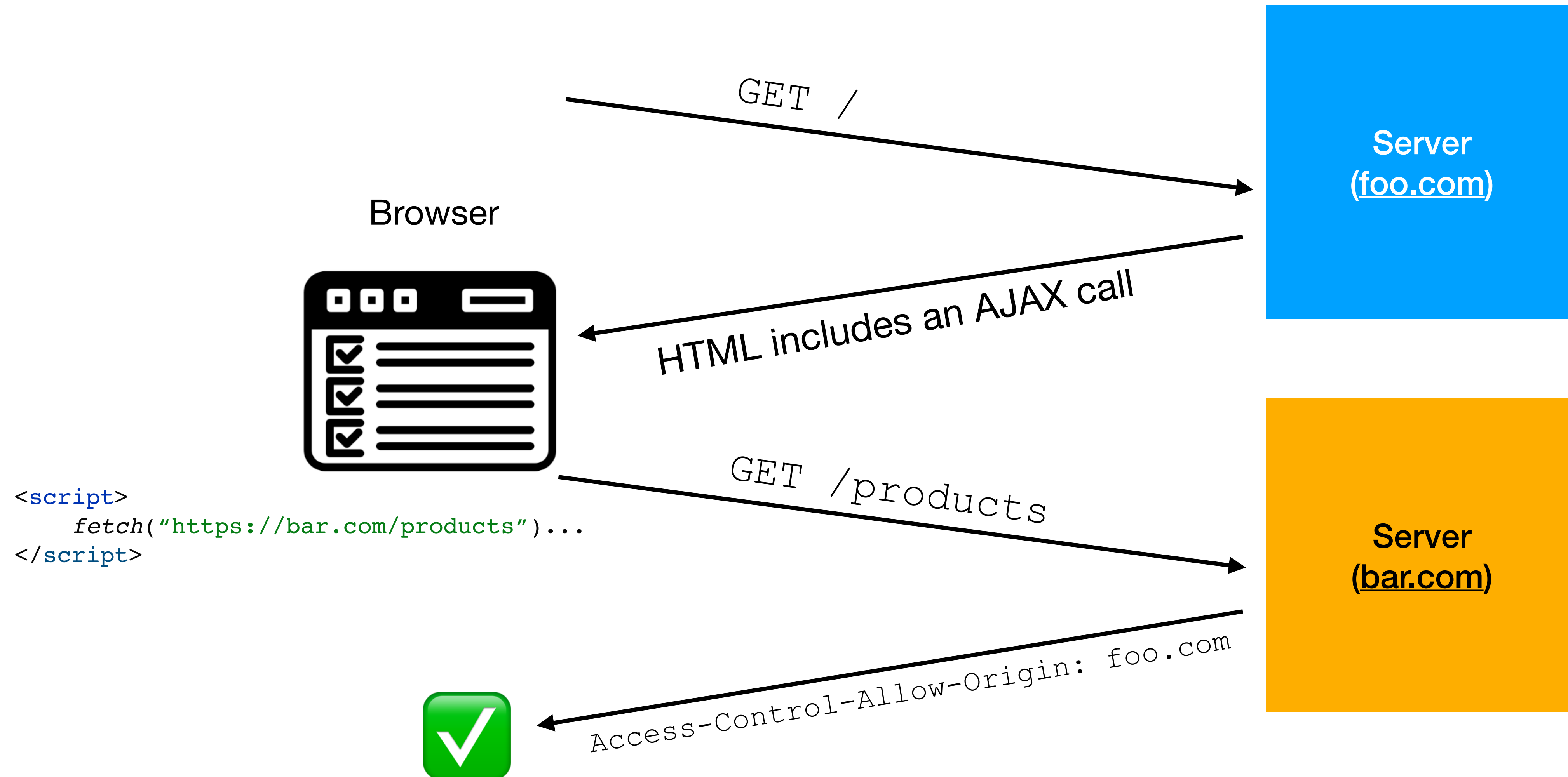
```
Access-Control-Allow-Origin: <allowed sites>
```

CORS - Cross-Origin Resource Sharing



✗ Access to fetch at 'https://bar.com/products' from origin 'https://foo.com' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

CORS - Cross-Origin Resource Sharing



Exercise

	Web Application	Public API	Internal API (*)
Authentication (Form-based, Token-based, OAuth 2, N/A)			
Session (Cookie-based, Stateless)			
CORS (Allowed, Forbidden, N/A)			
Response format (HTML full, HTML partial, JSON)			
Client			

(*) For frontend consumption

Resolution

	Web Application	Public API	Internal API
Authentication (Form-based, Token-based, OAuth 2, N/A)	Form-based login / OAuth 2	Token based / OAuth 2	N/A
Session (Cookie-based, Stateless)	Cookie-based	Stateless	Cookie-based
CORS (Allowed, Forbidden, N/A)	N/A	Usually allowed	Forbidden
Response format (HTML full, HTML partial, JSON)	HTML (full pages)	JSON	HTML (partials) or JSON
Client	Browser	Browser / Mobile / Other servers	Browser

The DOM as a Stateful Data Structure

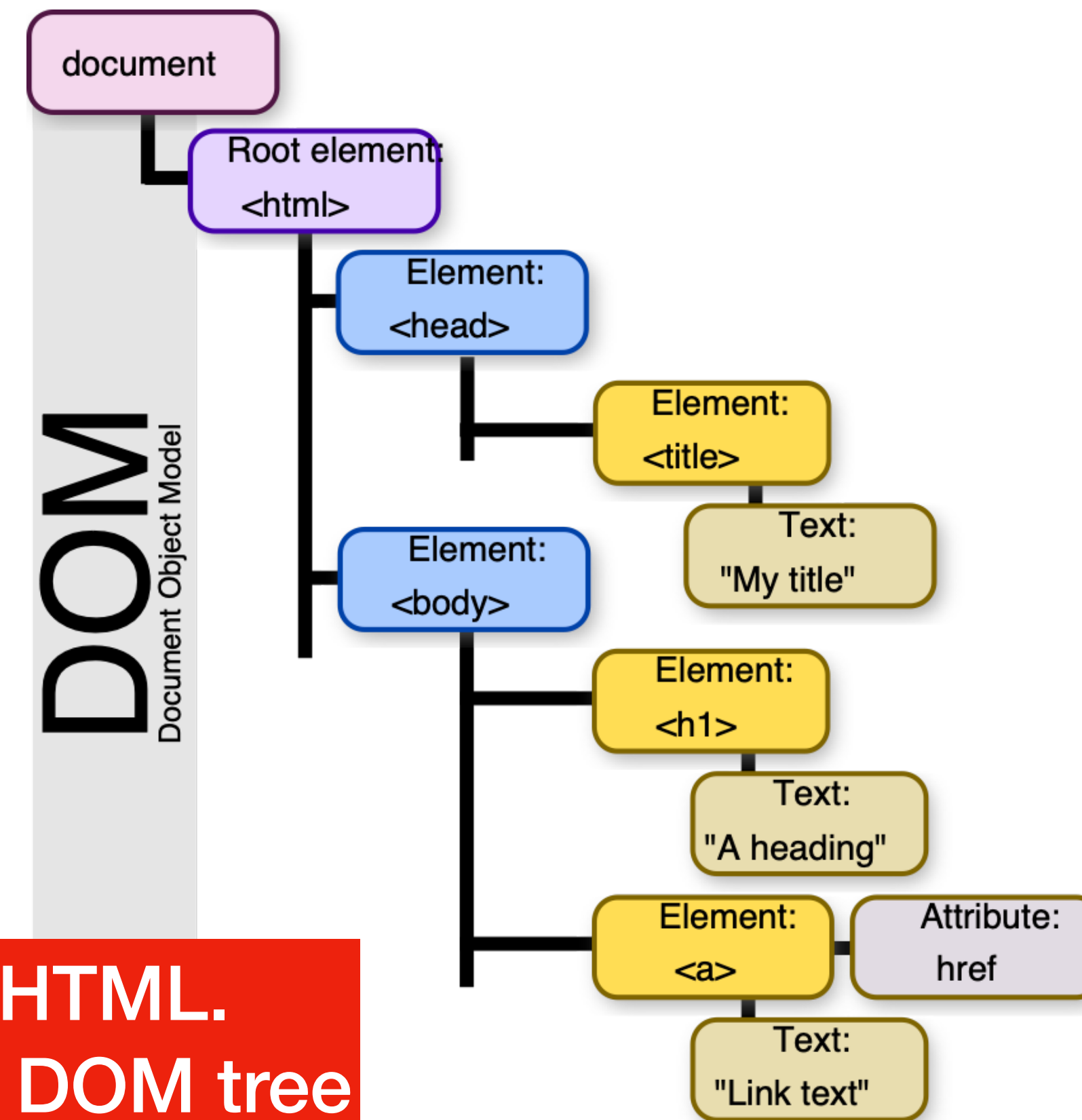
The DOM (Document Object Model) is a tree-structured, in-memory representation of a document

Nodes:

- elements
- text
- attributes

It is **mutable** and **observable**

**The browser does not render HTML.
It renders the current state of the DOM tree**



The DOM as a Mutable Data Structure

The DOM can be mutated:

Structural - add and remove nodes

`appendChild, remove`

Attribute - modify node properties

`class, style`

Content - update text/HTML

`textContent, innerHTML`

DOM updates trigger incremental redraw of affected parts of the page

The DOM as a Mutable Data Structure

```
<!DOCTYPE html>
<html>
<body>

  <h3>1. Add / Remove Nodes</h3>
  <ul id="list">
    <li>Item 1</li>
  </ul>
  <button onclick="addItem()">Add</button>
  <button onclick="removeItem()">Remove</button>

  <h3>2. Modify Properties</h3>
  <p id="box">Hello</p>
  <button onclick="changeStyle()">Change style</button>

  <h3>3. Update Text / HTML</h3>
  <p id="text">Old text</p>
  <button onclick="updateText()">Update</button>

</body>
</html>
```

```
<script>
  function addItem() {
    const li = document.createElement("li");
    li.textContent = "New item";
    document.getElementById("list").appendChild(li);
  }

  function removeItem() {
    const list = document.getElementById("list");
    if (list.lastElementChild) {
      list.removeChild(list.lastElementChild);
    }
  }

  function changeStyle() {
    const el = document.getElementById("box");
    el.style.color = "red";
  }

  function updateText() {
    document.getElementById("text")
      .textContent = "New text";
  }
</script>
```

Dynamic frontend in SSR

Many times, the browser fetches data to drive DOM Mutations

Techniques:

- AJAX/Fetch
- HTML-over-the-wire (HTMX, Hotwire)
- Islands Architecture (partial hydration)

Dynamic frontend in SSR

AJAX/Fetch

Asynchronous requests from the browser to the server without reloading the page

```
fetch("/api/messages")
```

non-blocking (async) call, returns a `Promise<Response>`

The `Response` object has:

- `response.ok` — true if status is 2xx
- `response.status` — the HTTP status code
- `response.json()` — returns a `Promise<any>` (parses body as JSON)
- `response.text()` — returns a `Promise<string>`

Dynamic frontend in SSR

AJAX/Fetch

.then() chaining

```
document.addEventListener("DOMContentLoaded", () => {  
  fetch("/api/message")  
    .then(res => res.json())  
    .then(data => {  
      document.getElementById("output").textContent = data.message;  
    })  
    .catch(err => console.error("Failed:", err));  
});
```

Pipeline of steps

async/await

```
document.addEventListener("DOMContentLoaded", async () => {  
  try {  
    const res = await fetch("/api/message");  
    const data = await res.json();  
    document.getElementById("output").textContent = data.message;  
  } catch (err) {  
    console.error("Failed:", err);  
  }  
});
```

Looks like synchronous code
Must execute in a async function

Dynamic frontend in SSR

AJAX/Fetch

```
<button type="button" id="getInfoBtn">Get Info</button>
```

```
<div id="result"></div>
```

Clicking this button will fetch data from the server and fill in the result



```
<script>
  const result = document.getElementById('result');
  document.getElementById('getInfoBtn').addEventListener('click', function () {
    fetch('/api/info')
      .then(r => r.json())
      .then(data => {
        result.innerHTML = data.info;
      })
      .catch(() => result.innerHTML = 'Failed to fetch info');
  });
</script>
```


Dynamic frontend in SSR

HTML-over-the-wire

Server returns HTML fragments instead of JSON

Minimal use of Javascript

Compliant with REST principles

REST allows full or partial HTML view responses

Several libraries available: HTMX, Hotwire, ...

Dynamic frontend in SSR

HTML-over-the-wire

Benefits

- Fast first-load pages (when compared to CSR)
- Keeps template rendering on the server
- Enhances SSR with interactive frontends without requiring Javascript

Dynamic frontend in SSR

HTMX

- Lightweight JS library
- Extends HTML with attributes like:
 - `hx-get`, `hx-post`
 - `hx-target`
 - `hx-swap`



Dynamic frontend in SSR

HTMX

```
<button hx-get="/hello" hx-target="#result">
  Click me
</button>

<div id="result"></div>
```

When you click the button:

1. AJAX call to /hello
2. Server returns HTML fragment
3. Swaps the content of the “result” element with the fragment

Need to include this:

```
<script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.4/dist/htmx.min.js"></script>
```

Dynamic frontend in SSR

HTMX

```
<button hx-put="/messages">
  Put To Messages
</button>
```

When a user clicks on this button, issue a PUT request to the URL /messages and load the response into the button

```
<div hx-post="/mouse_entered"
  hx-trigger="mouseenter"
  hx-vals='{ "location": "Lisbon" }'>
  [Here Mouse, Mouse!]
</div>
```

When the mouse enters this div, issue a POST request (with a payload) to the URL /mouse_entered and load the response into the div

```
<input type="text" id="name">...</input>
<button hx-get="/users"
  hx-include="#name">
  Get Users
</button>
```

Include the name as a query parameter, based on the value of the element “name”

Dynamic frontend in SSR

HTMX

+ Add Item

1

New item...

2

New item...

3

New item...

Can also be used to include dynamic content that was rendered in the server

```
<button class="btn btn-primary"
  hx-get="/todo/item"
  hx-target="#todoList"
  hx-swap="beforeend">+ Add Item</button>
```

```
<table class="table table-sm mb-0">
  <tbody id="todoList"></tbody>
</table>
```

fragment in the server

```
<tr th:fragment="todo-item">
  <td th:text="${index}"></td>
  <td>
    <input type="text" placeholder="New item..." />
  </td>
</tr>
```


Dynamic frontend in SSR

HTMX

Error Handling

- By default, 4xx and 5xx response codes are ignored (no swap occurs)
- It is possible to redefine this behaviours using a meta tag

```
<!--all responses are swapped-->  
<meta name="htmx-config" content='{ "responseHandling": [ { "code": ".*", "swap": true } ] }' />
```

- On errors, htmx triggers “htmx:responseError” events which can be handled in Javascript

Dynamic frontend in SSR

HTMX

There's a plugin for IntelliJ that adds support for HTMX auto-complete



<https://plugins.jetbrains.com/plugin/20588-htmx-support>

Dynamic frontend in SSR

Hotwire

Invented by the creators of Ruby on Rails

Extends HTML with turbo-frames: self-contained regions of HTML that can be updated independently without reloading the whole page

```
<turbo-frame>  
  <a href="/hello">Click me</a>  
</turbo-frame>
```

When you click the link:

1. AJAX call to /hello
2. Server returns HTML fragment (another turbo-frame)
3. Swaps the content of the originating turbo-frame with the fragment

Dynamic frontend in SSR

HTMX vs Hotwire

HTMX

```
<button hx-get="/hello" hx-target="#result">
  Click me
</button>
```

```
<div id="result"></div>
```

- **Behavior defined via attributes**
- Explicit request + explicit target
- Fine-grained DOM updates
- Any element can trigger requests

Hotwire

```
<turbo-frame>
  <a href="/hello">Click me</a>
</turbo-frame>
```

- **Behavior defined via HTML structure**
- Frame defines update boundary
- Whole region is replaced
- Uses standard links/forms

Dynamic frontend in SSR

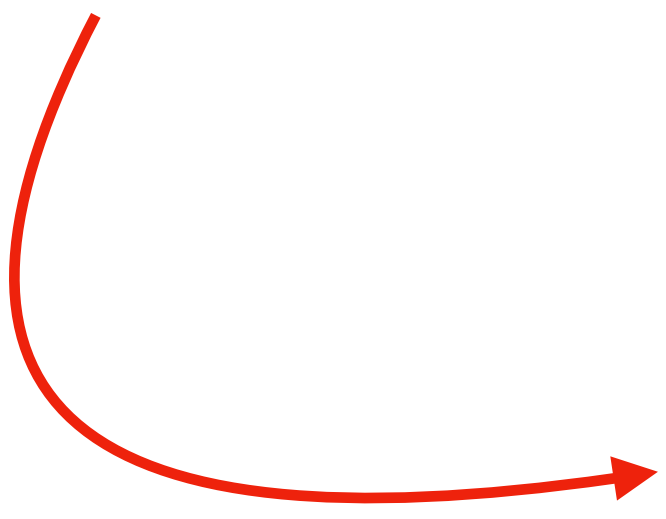
HTML Fragments on the Server

Server must return HTML Fragments

```
@Controller
@RequestMapping("/api")
class WeatherController(private val weatherService: WeatherService) {

    @GetMapping("/weather")
    fun checkWeatherHtml(@RequestParam location: String, model: ModelMap): String {
        model["raining"] = weatherService.isRaining(location)
        return "fragments/weather :: result"
    }
}
```

```
resources/
└─ templates/
    └─ fragments/
        └─ weather.html
```



```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
<span th:fragment="result" th:switch="${raining}">
    <span th:case="true">Raining</span>
    <span th:case="false">Clear</span>
    <span th:case="*">Weather data unavailable.</span>
</span>
</body>
</html>
```


JSON vs HTML Fragments on the Server

Server handles all AJAX requests the same way

It decides the output format based on content-negotiation headers in the HTTP request (Accept)

This must be set explicitly, client-side:

```
fetch('/api/weather?location=Paris', {  
  headers: {  
    "Accept": "application/json"  
  }  
})
```

Accept: application/json

```
<button hx-get="/api/weather?location=Paris"  
  hx-headers='{ "Accept": "text/html" }'>
```

Accept: text/html

AJAX vs HTML Fragments on the Server

Same endpoint ("/api/weather), different behaviours

```
@Controller
@RequestMapping("/api")
class WeatherController(private val weatherService: WeatherService) {

    @GetMapping("/weather", produces = [MediaType.APPLICATION_JSON_VALUE]) ← Accept: application/json
    @ResponseBody
    fun checkWeatherJson(@RequestParam location: String): Map<String, Any?> =
        mapOf("raining" to weatherService.isRaining(location))

    @GetMapping("/weather", produces = [MediaType.TEXT_HTML_VALUE]) ← Accept: text/html
    fun checkWeatherHtml(@RequestParam location: String, model: ModelMap): String {
        model["raining"] = weatherService.isRaining(location)
        return "fragments/weather :: result"
    }
}
```

Dynamic frontend in SSR

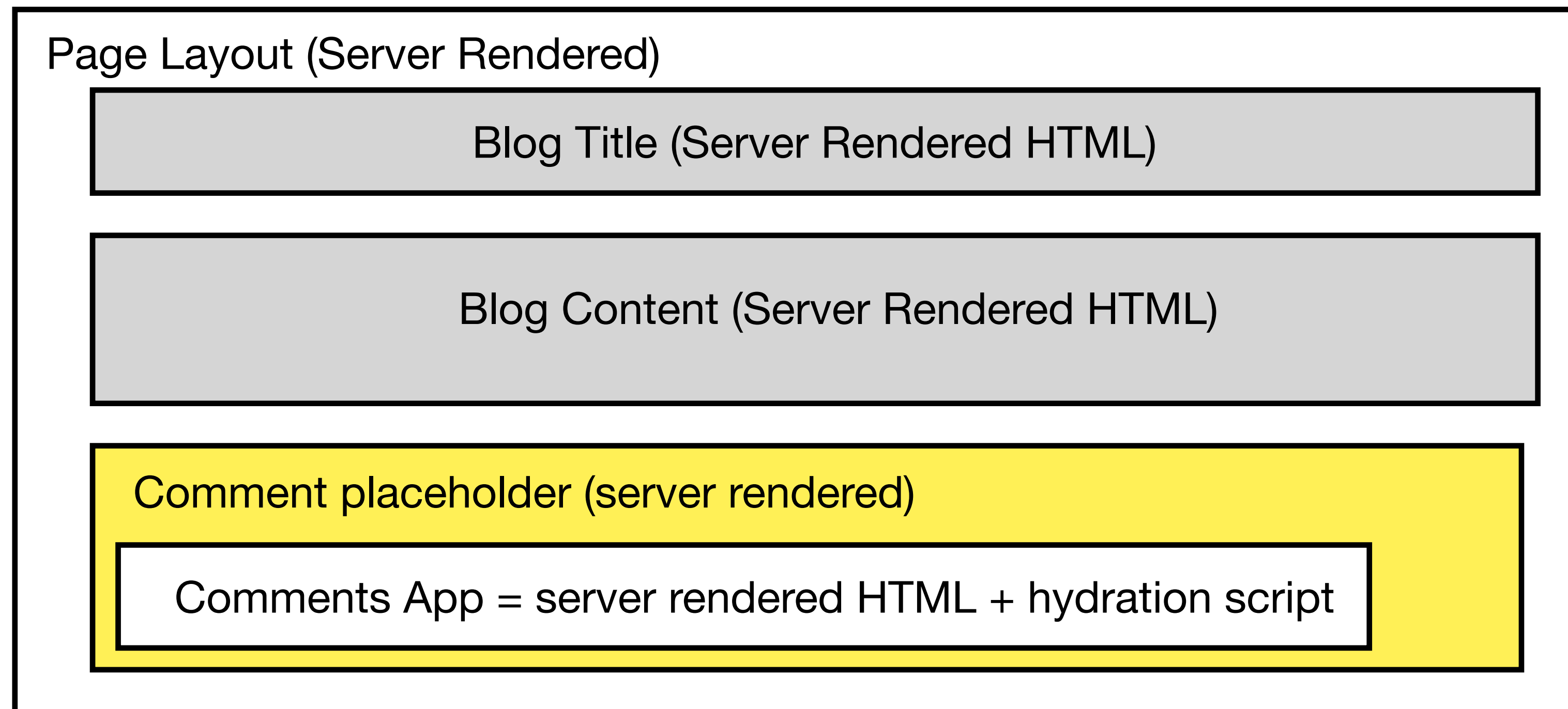
Techniques:

- AJAX/Fetch
- HTML-over-the-wire (HTMX, Hotwire)
- **Islands Architecture (partial hydration)**

Dynamic frontend in SSR

Islands Architecture

The page is rendered on the server but some dynamic components (islands) are rendered as just placeholders that will be filled in the browser by an hydration script



Dynamic frontend in SSR

Hydration

1. Server returns a placeholder and an hydration script

```
<div id="comments"></div>

<script>
  async function loadComments() {
    ...
  }
</script>
```



Fetches comments via AJAX
Swaps the content of the placeholder

2. Browser runs the hydration script, filling in the placeholder

```
<div id="comments">
  <div class="comment"><strong>Alice</strong>: Hello!</div>
  <div class="comment"><strong>Bob</strong>: Nice post</div>
</div>
```


Dynamic frontend in SSR

Rehydration

1. Server returns the placeholder already filled in and a rehydration script

```
<div id="comments">
  <div class="comment"><strong>Alice</strong>: Hello!</div>
  <div class="comment"><strong>Bob</strong>: Nice post</div>
</div>

<script>
  async function loadComments() {
    ...
  }
</script>
```

2. Browser runs the rehydration script, updating the placeholder

```
<div id="comments">
  <div class="comment"><strong>Alice</strong>: Hello!</div>
  <div class="comment"><strong>Bob</strong>: Nice post</div>
  <div class="comment"><strong>Mary</strong>: Hi, there!</div>
</div>
```

Hydration

Case study: Datatables.net

10 ▾

 entries per page

Search:

Name	Position	Office	Age
▶ Airi Satou	Accountant	Tokyo	33
▶ Angelica Ramos	Chief Executive Officer (CEO)	London	47
▶ Ashton Cox	Junior Technical Author	San Francisco	66
▶ Bradley Greer	Software Engineer	London	41
▶ Brenden Wagner	Software Engineer	San Francisco	28
▶ Brielle Williamson	Integration Specialist	New York	61
▶ Bruno Nash	Software Engineer	London	38
▶ Caesar Vance	Pre-Sales Support	New York	21
▶ Cara Stevens	Sales Assistant	New York	46
▶ Cedric Kelly	Senior JavaScript Developer	Edinburgh	22

NamePositionOfficeAge

Showing 1 to 10 of 57 entries

« < 1 2 3 4 5 6 > »

Advanced interactive tables

Built-in sorting, searching, pagination, etc...

Supports hydration and rehydration

Hydration

Case study: Datatables.net

Hydration - Server returns empty usersTable

```
<table id="usersTable" class="display">
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>City</th>
    </tr>
  </thead>
</table>

<script src="https://.../dataTables.min.js"></script>
<script>
  new DataTable('#usersTable', {
    ajax: '/api/users',
    columns: [
      { data: 'id' },
      { data: 'name' },
      { data: 'city' }
    ]
  });
</script>
```

Rehydration - Server returns filled-in usersTable

```
<table id="usersTable" class="display">
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>City</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="u : ${users}">
      <td th:text="${u.id}"></td>
      <td th:text="${u.name}"></td>
      <td th:text="${u.city}"></td>
    </tr>
  </tbody>
</table>

<script src="https://.../dataTables.min.js"></script>
<script>
  new DataTable('#usersTable', {
    ajax: '/api/users',
    columns: [
      { data: 'id' },
      { data: 'name' },
      { data: 'city' }
    ]
  });
</script>
```


Exercise

	AJAX/Fetch	HTMX	Islands Architecture
Triggering data fetching (Automatic, Declarative, Programmatic)			
Data format (JSON/HTML)			
Rendering location (Client/Server)			
Typical interaction model (triggered by user, automatic)			
Complexity (high, medium, low)			

Resolution

	AJAX/Fetch	HTMX	Islands Architecture
Triggering data fetching (Automatic, Declarative, Programmatic)	Programmatic	Declarative	Automatic (on page load)
Data format (JSON/HTML)	JSON	HTML	JSON (usually)
Rendering location (Client/Server)	Client	Server	Both
Typical interaction model (triggered by user, automatic)	Triggered by the user (event-driven)	Triggered by the user (event-driven)	Automatic (hydration)
Complexity (high, medium, low)	Medium	Low	Medium

Automated Testing



Automated Testing

Automated Test

A small piece of code that validates a specific feature of your app

Given a set of actions (along with data), does the application produce the expected results?

Once written, a test can be run repeatedly with minimal effort.

Automated Testing

Why?

- Make sure a feature works as expected under a variety of conditions
- Document features in a way that stays accurate and up to date
- Enable robust development practices (TDD)
- Improve software design and code quality
- Provide a safety net for future changes (bug fixes, refactoring, etc...)
- Support automation pipelines that minimize time from commit to deploy (CI/CD)

Automated Testing

Automated Testing is still (nowadays) not as prevalent as it should be

Across 82k OSS projects, only 17% include automated tests

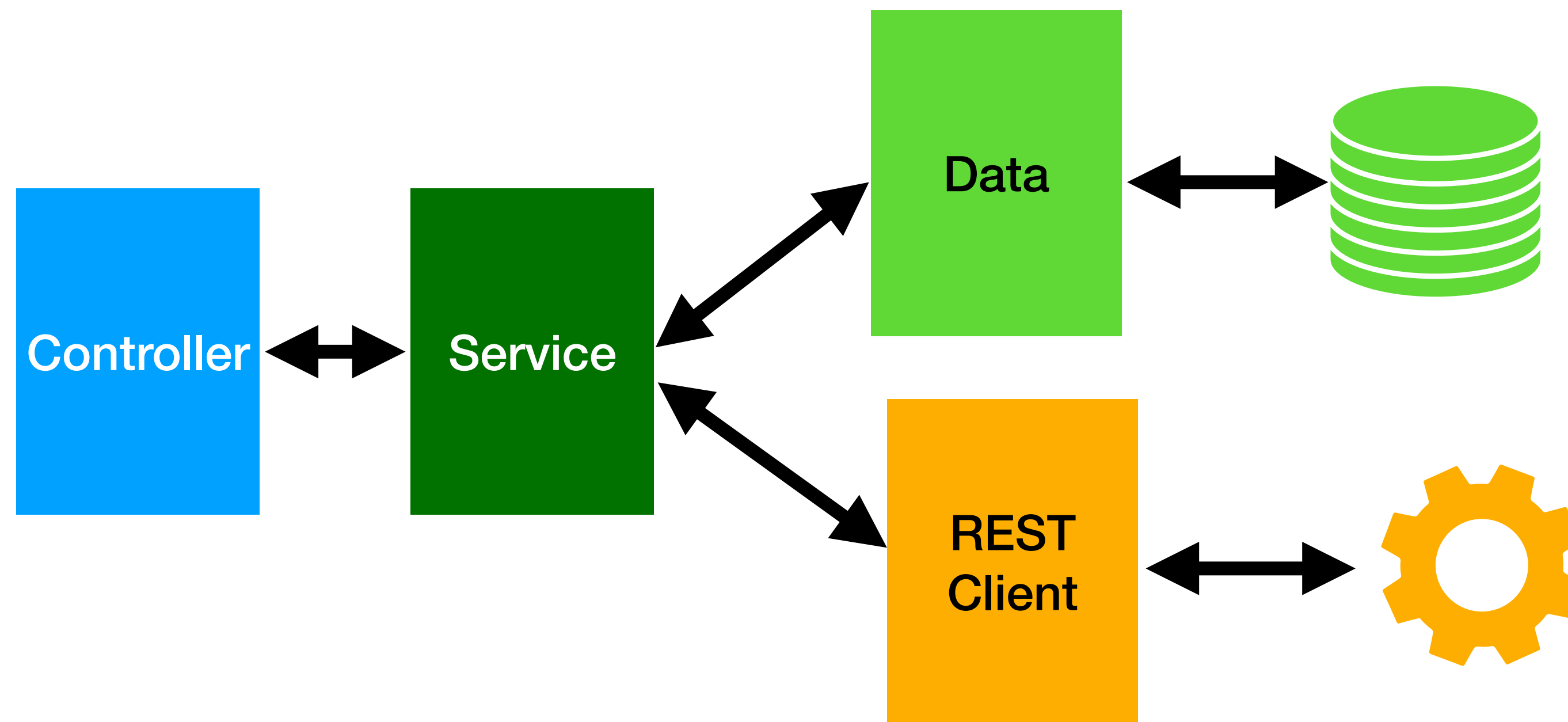
Gonzalez, Danielle, et al. "A large-scale study on the usage of testing patterns that address maintainability attributes: patterns for ease of modification, diagnoses, and comprehension." 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR). IEEE, 2017.

Across 9k OSS Deep-Learning projects, only 32% include automated tests

Wang, Han, et al. "Beyond accuracy: An empirical study on unit testing in open-source deep learning projects." ACM Transactions on Software Engineering and Methodology 33.4 (2024): 1-22.

Automated Testing

Testing Internet Applications implies testing multiple components and their interactions



Automated Testing

Types of Tests

Feature	Unit Tests	Integration Tests
Scope	Single class/method	Multiple components together
Execution Speed	Very fast	Slower
Dependencies	None/Mocked/Faked	Mocked/Faked/Real
Purpose	Test logic	Test system behavior
Example	Test a calculator class (add, subtract)	Test an interaction with a web page (login, get list, get detail)

Automated Testing

Spring

Spring is designed with automated testing in mind:

- Built-in Dependency Injection (DI) improves testability
- Rich testing ecosystem (e.g., JUnit, Mockito, Spring Test)
- Supports integration testing out of the box (e.g., `@SpringBootTest`)

Automated Testing

Spring - Maven

```
project-root/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/example/app/
│   │   │   │   ├── controller/
│   │   │   │   ├── service/
│   │   │   │   └── repository/
│   │   └── resources/
│   │       └── application.properties
│   └── test/
│       ├── java/
│       │   ├── com/example/app/
│       │   │   ├── controller/
│       │   │   ├── service/
│       │   │   └── repository/
│       └── resources/
│           └── application-test.properties
```

Tests should live in the `src/test` folder and mirror the packages of the classes being tested

In Spring, tests run with the “test” profile by default, so `application-test.properties` **overrides** `application.properties`

Automated Testing

Spring - Maven

Add these dependencies

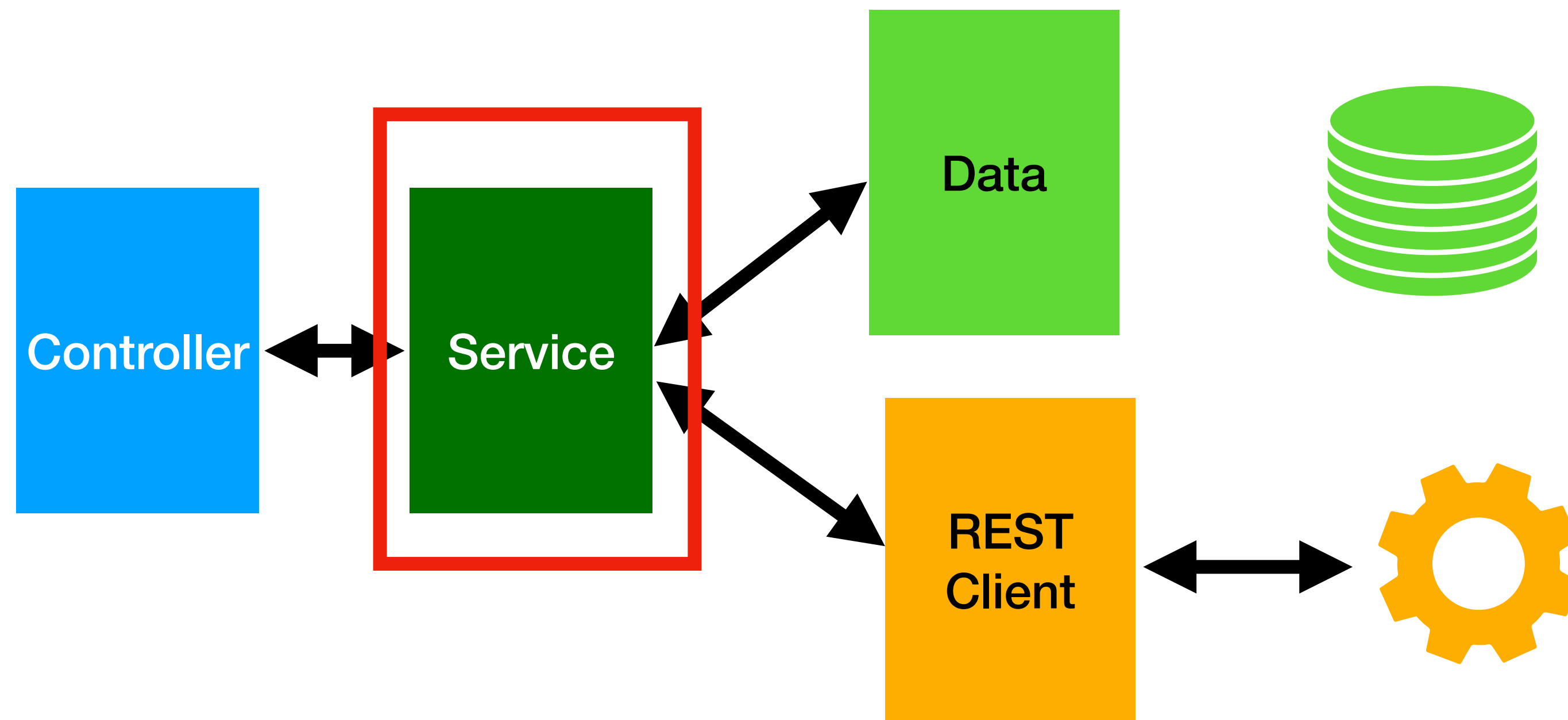
```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-test</artifactId>  
  <scope>test</scope>  
</dependency>
```

```
<dependency>  
  <groupId>org.springframework.security</groupId>  
  <artifactId>spring-security-test</artifactId>  
  <scope>test</scope>  
</dependency>
```

← You'll need this to test
authenticated endpoints

Spring Testing

Testing Services



Spring Testing

Unit Tests

```
@Service
class CalculatorService {
    fun add(a: Int, b: Int) = a + b
}
```

Plain Old JUnit Test

No dependency on Spring

```
class CalculatorServiceTest {

    val calculatorService = CalculatorService()

    @Test
    fun `when add 1 + 1 it should return 2`() {
        assertEquals(2, calculatorService.add(1, 1))
    }

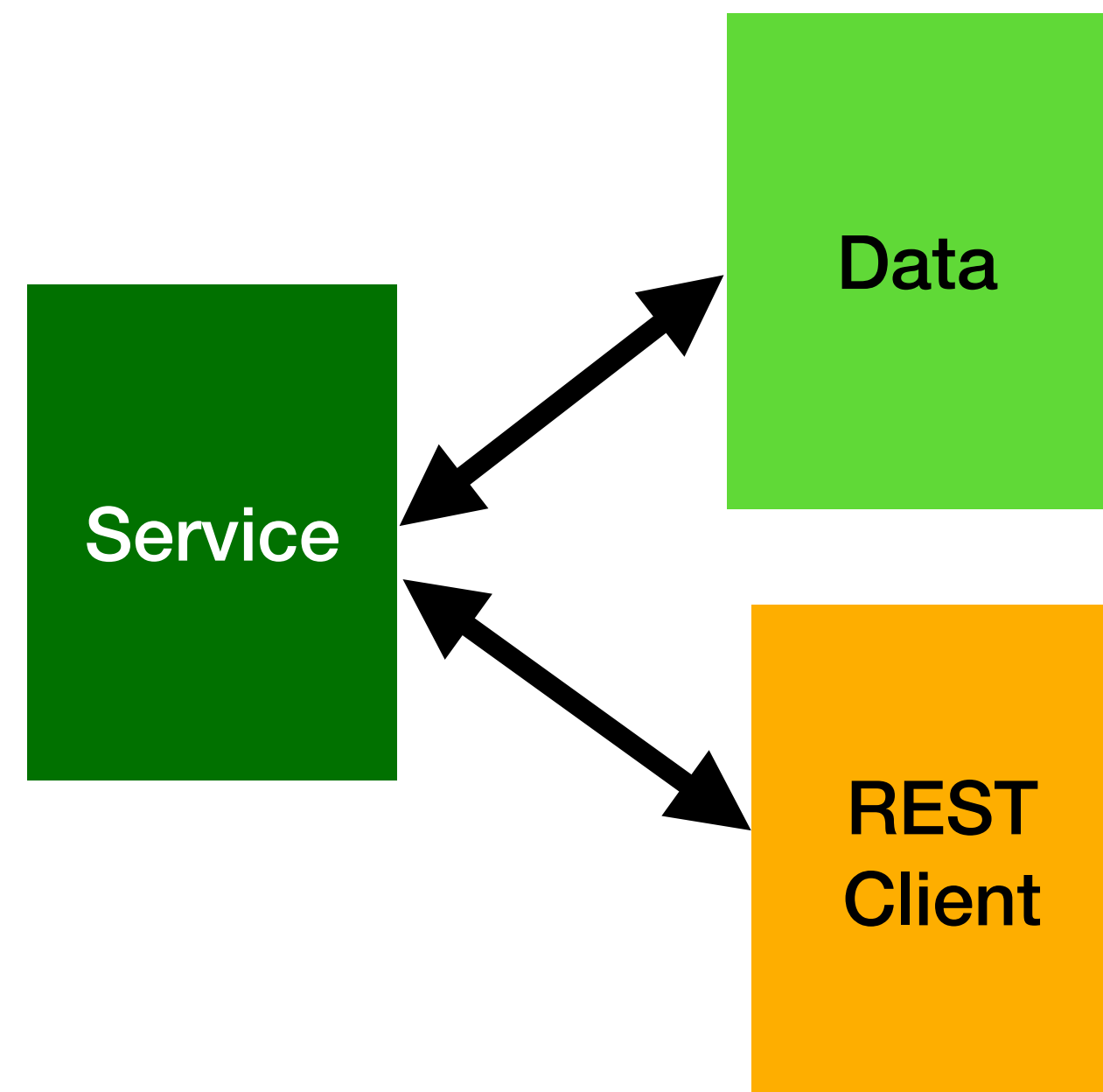
}
```

Spring Testing

Service Unit Tests

```
@Service
class UserService(val userRepository: UserRepository) {
    fun getUser(username: String) = userRepository.findByUsername(username)
}
```

Usually the services depend on repositories and or REST clients (also, other services)



Spring Testing

Unit Tests with Mocks

```
@ExtendWith(MockitoExtension::class)
class UserServiceTests {

    @Mock
    lateinit var repo: UserRepository

    @InjectMocks
    lateinit var service: UserService

    @Test
    fun `getByUsername returns user when found`() {
        `when`(repo.findByUsername("alice"))
            .thenReturn(User(id = 1, username = "alice"))

        val result = service.getUser("alice")

        assertEquals("alice", result.username)
    }
}
```

Unit test with mocked
dependencies injected

No database involved

Spring Testing

Unit Tests with Fakes

```
class FakeUserRepository : UserRepository {
    private val store = mutableMapOf<Long, User>()
    private var nextId = 1L

    override fun findByUsername(username: String): User? =
        store.values.find { it.username == username }

    override fun <S : User> save(entity: S): S {
        ...
    }
}
```

```
class UserServiceTests {

    private val service: UserService = UserService(FakeUserRepository())

    @Test
    fun `getByUsername returns user when found`() {
        service.create("alice")

        val result = service.getUser("alice")

        assertEquals("alice", result.username)
    }
}
```

Unit test with faked
dependencies injected

No database involved

Spring Testing

Mocks vs Fakes

Mock

- Created dynamically by a framework (e.g., Mockito)
- You define expected interactions in the test
- Verifies how the dependency is used
- Faster to develop, harder to debug

```
@Mock
lateinit var repo: UserRepository

`when` (repo.findByUsername("alice"))
    .thenReturn(User(id = 1, username = "alice"))
```

Fake

- A real implementation written for testing
- Often uses simplified logic (e.g., in-memory storage)
- Injected like a normal dependency
- More effort to write, easier to debug

```
val service = UserService(FakeUserRepository())
```

Spring Testing

Integration Tests

`@SpringBootTest` ← This creates the Spring context

```
class UserServiceIntegrationTest {
```

```
    @Autowired
```

```
    lateinit var userService: UserService
```

```
    @Test
```

```
    fun `createUser saves to DB and getUser returns it`() {
```

```
        userService.createUser("alice")
```

```
        val loaded = userService.getUser("alice")
```

```
        assertNotNull(loaded)
```

```
        assertEquals("Alice", loaded!!.username)
```

```
    }
```

```
}
```

Integration test using a
real database (*)

(no mocking/faking)

(*) There is no application-test.properties so it uses the one in application.properties

Spring Testing

Integration Tests

```
@SpringBootTest
class UserServiceIntegrationTest {

    @Autowired
    lateinit var userService: UserService

    @Test
    fun `createUser saves to DB and getUser returns it`() {
        userService.createUser("alice")
        val loaded = userService.getUser("alice")

        assertNotNull(loaded)
        assertEquals("Alice", loaded!!.username)
    }
}
```

src/test/resources/application-test.properties

```
# configuration for H2 embedded in-memory database
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
```

Integration tests should
use their own database

(preferably in-memory to be faster)

(no mocking/faking)

Spring Testing

Integration Tests

```
@SpringBootTest
class UserServiceIntegrationTest {

    @Autowired
    lateinit var userService: UserService

    ...
}
```

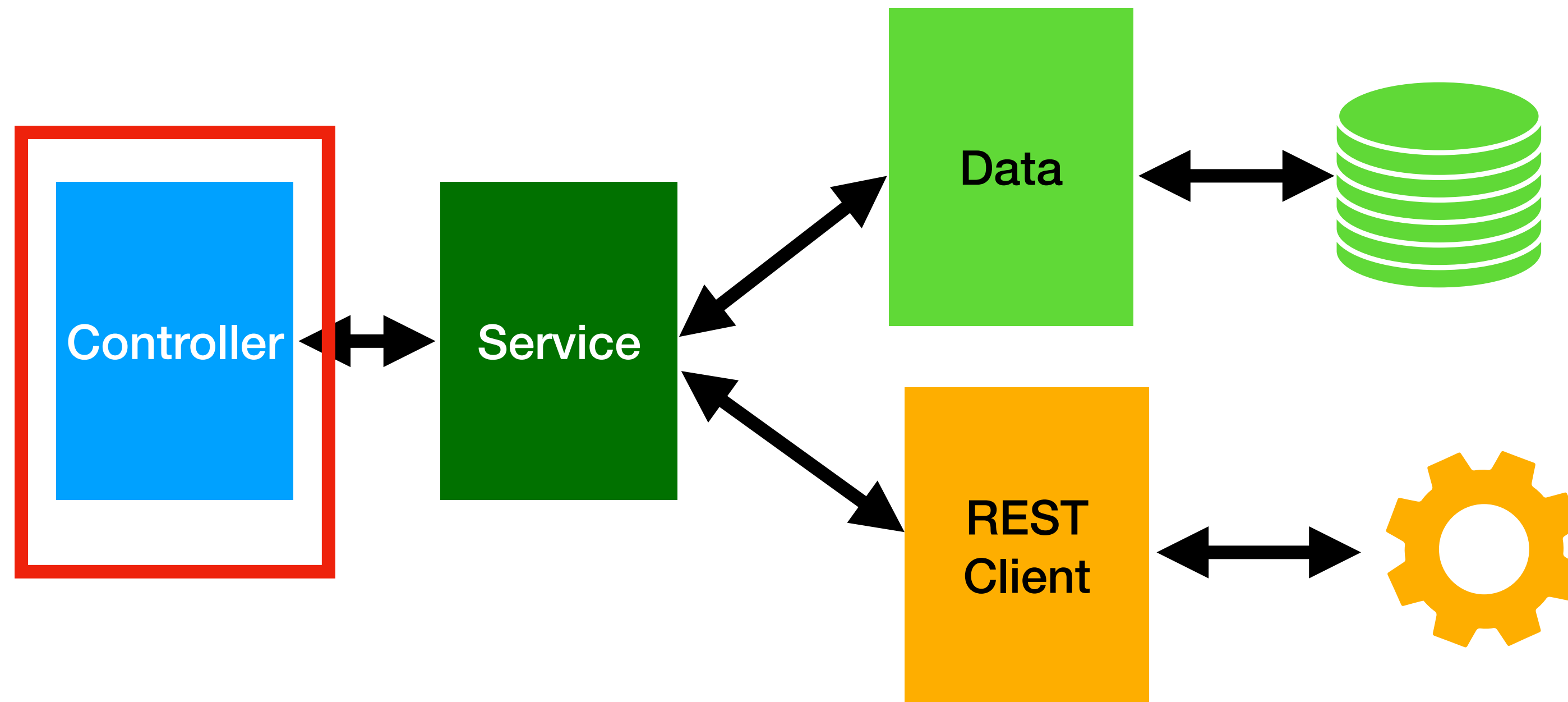


Starts a full Spring application

- Creates all beans
- Enable dependency injection
- Loads external configurations (application.properties)
- ...

Integration Tests - Controllers

Usually, Controllers are tested by Integration Tests, backed by Spring's MockMvc



Spring Testing

Integration Tests - Controllers

```
@SpringBootTest
@AutoConfigureMockMvc
class TodoMockMvcTests {
```



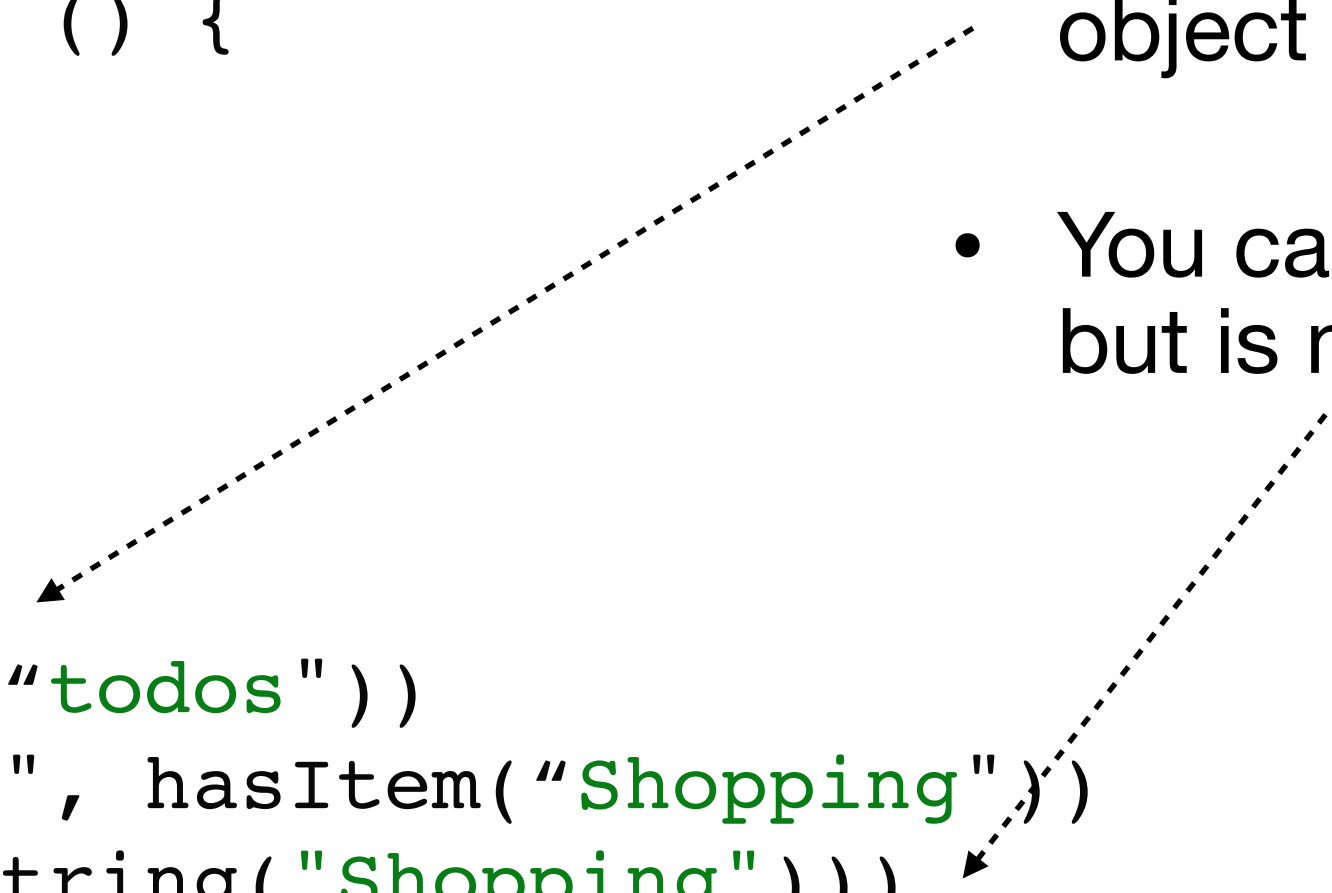
```
    @Autowired
    lateinit var mvc: MockMvc
```

```
    @Test
    fun `add a todo item and verify it is shown`() {
        mvc.perform(post("/todo")
            .param("text", "Shopping"))
            .andExpect(status().isOk)

        mvc.perform(get("/"))
            .andExpect(status().isOk)
            .andExpect(model().attributeExists("todos"))
            .andExpect(model().attribute("todos", hasItem("Shopping")))
            .andExpect(content().string(containsString("Shopping")))

    }
}
```

MockMvc is injected by Spring and acts as a browser (but no server is actually launched)

- You can assert directly the model object that was sent to the view
 - You can also inspect the html content but is more fragile
- 

Spring Testing

Integration Tests - Controllers

```
@RunWith(SpringRunner::class)
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class TodoRestTemplateTests {

    @Autowired
    lateinit var restTemplate: TestRestTemplate

    @Test
    fun `add a todo item and verify it is shown`() {
        val params = LinkedMultiValueMap<String, String>().apply {
            add("text", "Shopping")
        }
        val addResponse = restTemplate.postForEntity(
            "/todo",
            HttpEntity<LinkedMultiValueMap<String, String>>(params),
            String::class.java
        )
        assertEquals(HttpStatus.OK, addResponse.statusCode)

        val getResponse = restTemplate.getForEntity("/", String::class.java)
        assertEquals(HttpStatus.OK, getResponse.statusCode)
        assertTrue(getResponse.body!!.contains("Shopping"))
    }
}
```

With Random Port, an actual server is launched during the test

- No longer possible to use MockMvc
- You can use TestRestTemplate or a real http client (in that case, you have to build the whole url: http://localhost:port/todo)
- No longer possible to assert the model, you have to inspect the html
- But the Spring Context is shared between the test and the launched server

Spring Testing Integration Tests

The most interesting integration tests in Spring simulate a browser

Feature	<code>SpringBootTest.WebEnvironment.MOCK</code>	<code>SpringBootTest.WebEnvironment.RANDOM_PORT</code>
Server started?	No real server	Real embedded server
HTTP Stack	Fake (MockMvc)	Real HTTP
Speed	Fast	Slower
Use with	MockMvc / WebClient	TestRestTemplate / WebClient / any HTTP client
Possible to debug in browser (*)	No	Yes

(*) Put a breakpoint in the test and access localhost in the browser

Spring Testing

Integration Tests - Browser

```
import org.htmlunit.WebClient
...

@SpringBootTest
@AutoConfigureMockMvc
class GeneralWebClientTests {

    @Autowired
    lateinit var webClient: WebClient

    @Test
    @WithMockUser("student1", roles = ["STUDENT"])
    fun `student navbar shows correct items`() {
        val page: HtmlPage = webClient.getPage("/")

        // check username is displayed
        val usernameElement = page.querySelector<HtmlElement>("#currentUsername")
        assertEquals("student1", usernameElement.asNormalizedText())

        // check student-visible menu items are present
        assertNotNull(page.querySelector<HtmlElement>("#menuAccessTokens"))
        assertNotNull(page.querySelector<HtmlElement>("#menuMyHistory"))

        // check teacher/admin menu items are not present
        assertNull(page.querySelector<HtmlElement>("#navbarDropdownAssignments"))
        assertNull(page.querySelector<HtmlElement>("#navbarDropdownStudents"))
        assertNull(page.querySelector<HtmlElement>("#navbarDropdownAdmin"))
    }
}
```

```
<dependency>
  <groupId>org.htmlunit</groupId>
  <artifactId>htmlunit</artifactId>
  <scope>test</scope>
</dependency>
```

Testar a apresentação usando HtmlUnit

- **Web pages & content** — verify HTML, text, links, and page rendering
- **User interactions** — fill forms, click buttons, submit requests, navigate between pages
- **Basic JavaScript flows** — DOM updates, redirects, simple AJAX behavior

Pode ser usado com Mock ou com Random_Port

Spring Testing

Integration Tests

- MockMvc - request-based testing
- WebClient (HtmlUnit) - light browser-based testing
- Selenium - automate a real browser

Spring Testing

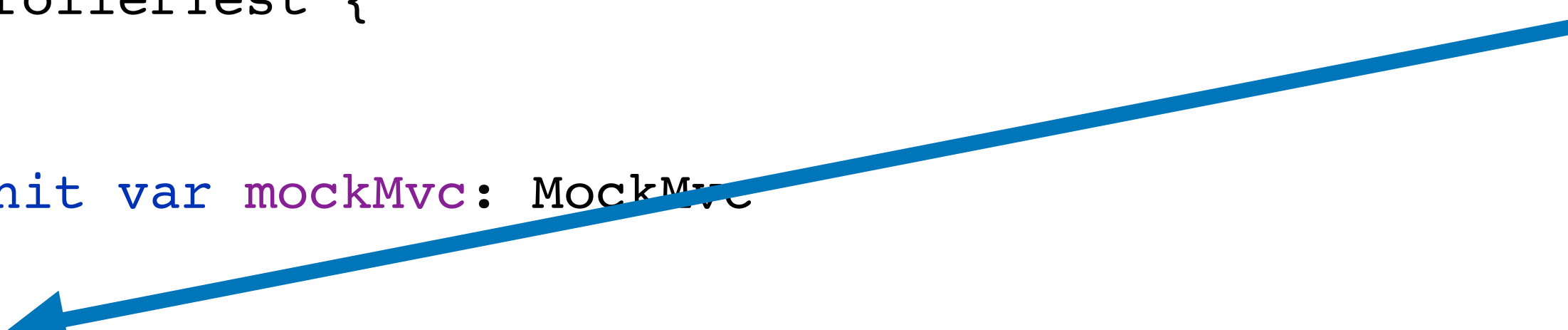
Integration Tests with Authentication

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class AirlineControllerTest {

    @Autowired
    private lateinit var mockMvc: MockMvc

    @Test
    @WithMockUser(username = "staff", roles = ["AIRLINE_REPRESENTATIVE"])
    fun `getAll returns list of airlines when authenticated`() {
        mockMvc.perform(get("/airlines"))
            .andExpect(status().isOk)
            .andExpect(content().contentType(MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$[0].id").value(1))
            .andExpect(jsonPath("$[0].iataCode").value("TP"))
            .andExpect(jsonPath("$[0].name").value("TAP Air Portugal"))
            ....
    }
}
```

Authentication and Authorization are conveniently handled by @WithMockUser



This bypasses the SecurityFilterChain and injects a Principal with this name and roles in the SecurityContext

Spring Testing

Integration Tests

Checklist

- Add dependencies for tests: spring-boot-starter-test and spring-security-test
- Add dependency for h2 (embedded DB)

```
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
  <scope>test</scope>  
</dependency>
```

- Use @SpringBootTest and @AutoConfigureMockMvc
- Create a test/resources/application-test.properties with an embedded DB

```
spring.datasource.url=jdbc:h2:mem:novaevents;DB_CLOSE_DELAY=-1  
spring.datasource.driver-class-name=org.h2.Driver  
spring.datasource.username=sa  
spring.datasource.password=
```

Spring Testing

Test Coverage

How much of your code is tested by automated tests?

Code tree

File list

[drop-project](#) / [src](#) / [main](#) / [kotlin](#) / [org](#) / [dropProject](#)

All flags

Search for files

Files ↑	Tracked lines	Covered	Partial	Missed		Coverage %
..						
 controllers	1359	945	170	244		69.54%
 dao	378	343	16	19		90.74%
 data	414	364	23	27		87.92%
 extensions	20	11	4	5		55.00%
 filters	10	5	5	0		50.00%
 forms	36	36	0	0		100.00%
 repository	2	2	0	0		100.00%
 security	70	60	5	5		85.71%
 services	1575	1238	162	175		78.60%
 storage	29	17	4	8		58.62%

Spring Testing

Test Coverage

There are several maven plugins that calculate coverage
jacoco is the most popular

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.14</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>generate-code-coverage-report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

After running `mvn test`, you can
check results at

`target/site/jacoco`

Spring Testing

Test Coverage

The screenshot displays the IntelliJ IDEA interface. On the left, the Project Explorer shows a directory structure: test > kotlin > pt > unl > fct > iadi > novaevents. The 'novaevents' folder is selected. In the center, the 'Run' menu is open, listing various actions. The 'More Run/Debug' option is highlighted, which has opened a sub-menu on the right. This sub-menu contains three options: 'Run \'Tests in \'novaevents\' with Coverage' (highlighted with a red box), 'Profile \'Tests in \'novaevents\' with \'IntelliJ Profiler\'', and 'Modify Run Configuration...'. The background of the IDE shows a code editor with a search bar and navigation bar.

Action	Shortcut
Paste	⌘V
Paste from History...	⇧⌘V
Find Usages	⇧⌘F7
Find in Files...	⇧⇧F
Replace in Files...	⇧⇧R
Analyze	>
SpotBugs	>
Rename...	⇧F6
Refactor	>
Bookmarks	>
Reformat Code	⇧⌘L
Optimize Imports	⇧⌘O
Delete...	⌘X
Rebuild 'pt.unl.fct.iadi.novaevents'	⇧⌘F9
Run 'Tests in 'novaevents''	⇧⇧F10
Debug 'Tests in 'novaevents''	⇧⇧F9
More Run/Debug	>
Open In	>
Local History	>

Search Everywhere Double ⇧
Go to File ⇧⌘N
Recent Files ⌘E
Navigation Bar ⇧⌘N
Drop files here to open them

Run 'Tests in 'novaevents' with Coverage
Profile 'Tests in 'novaevents' with 'IntelliJ Profiler'
Modify Run Configuration...

Spring Testing

Test Coverage

- Helps find untested parts of your code
- Reduces risk of bugs in production
- Improves code reliability and maintainability

In real-world projects, 100% coverage is rarely achieved